

# SD Basic - Operating on Whole Arrays

Walking an array, changing the mark level, the s functions, sorting and mark mapping.

---

This page continues SD Basic - Dynamic Arrays.

---

## Walking an array

```
remove variable from array setting delimiter
```

`remove` takes the next element off the front of *array* and reports which mark followed it. It is much faster than repeated `extract()` on a long record, because it does not rescan from the start each time.

**The delimiter code is the mark's level, and zero means the end.** For `p^q|r\s`:

| CODE | MEANING                                       |
|------|-----------------------------------------------|
| 0    | end of the string — this was the last element |
| 2    | a field mark followed                         |
| 3    | a value mark followed                         |
| 4    | a subvalue mark followed                      |

```
s = record
loop
  remove element from s setting mark
  ...
while mark
repeat
```

`remove` **consumes the variable it reads**. Work on a copy — `s = record` — or the record is gone by the end of the loop. The functions that follow depend on an internal pointer into that variable:

---

`getrem(string)`

how far through the string the pointer has reached

---

---

`setrem offset on string``move the pointer`

---

After one `remove` from `aaa^bbb`, `getrem()` is `4`.

**Changing the string resets the pointer.** A loop that both `remove`s from a string and assigns to it will restart from the beginning, silently.

---

## Changing the mark level

```
raise(string)
lower(string)
```

`raise` promotes every mark one level — subvalues become values, values become fields. `lower` demotes them.

`lower('x' : @fm : 'y')` has one field and two values; `raise('x' : @vm : 'y')` has two fields.

These are how you move a nested structure between levels without walking it — for example turning a multivalued field into a record.

---

## Working across an array

`VSLICE`

```
vslice(array, value.number)
```

Takes the  $n$ -th value of **every** field and returns them as fields. On `a1|a2^b1|b2`, `vslice(h, 2)` gives `a2^b2`.

That is the idiom for pulling one column out of a set of associated multivalued fields.

`SUBSTRINGS`, `FIELDSTORE` AND `SPLICE`

---

`substrings(array, start, length)`

the substring operator applied to every element

---

`fieldstore(string, delimiter, start, count, replacement)`

replace or insert delimited fields — a *count* of zero inserts

---

```
splICE(array1, string, array2)
```

joins the arrays element by element with *string* between

```
splICE('a' : @vm : 'b', '-', '1' : @vm : '2')
```

 gives `a-1|b-2`.

#### SUBSTITUTE

```
substitute(array, old.list, new.list {, delimiter})
```

Replaces whole **elements** that match. Substituting `b` with `Z` in the values `a`, `b`, `c` gives `a|Z|c`.

#### REUSE

```
reuse(expression)
```

Makes a single value behave as though it repeated for every element of the other operand. **Without it, arithmetic only reaches the first element.**

#### EXPRESSION

#### RESULT

```
(1\|2\|3) + 10
```

```
11\|2\|3
```

```
(1\|2\|3) + reuse(10)
```

```
11\|12\|13
```

The first line is almost never what was intended, and it produces a plausible number rather than an error.

## The `s` functions

Most string and arithmetic functions have a partner ending in `s` that applies the operation to every element instead of to the whole string: `abss()`, `negs()`, `mods()`, `nums()`, `lens()`, `counts()`, `fields()`, `trims()`, `trimbs()`, `trimfs()`, `indexs()`, `soundexs()`, `spaces()`, `strs()`, `fmts()`, `iconvs()`, `oconvs()`, `folds()`, `cats()`.

**When two Arrays are different lengths, the shorter is treated as empty, not truncated.** With `1|2|3` and `10|20`:

#### CALL

#### RESULT

```
cats(p, q)
```

```
110\|220\|3
```

the third pairs with nothing

```
mods(p, q)
```

```
1\|2\|3
```

`mod(3, 0)` is `3` — no error

So a mismatch produces a full-length answer with quietly wrong elements in the tail. **Compare** `dcount()` **on both operands before relying on the result.**

## Logical functions

These take true/false values element by element. In SD, zero and the null string are false and anything else is true.

|                                                                                                                 |                                               |
|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| <code>not(expression)</code>                                                                                    | logical negation of a single value            |
| <code>nots(array)</code>                                                                                        | of every element                              |
| <code>ands(a, b)</code> · <code>ors(a, b)</code>                                                                | element-by-element AND and OR                 |
| <code>eqs</code> · <code>nes</code> · <code>gts</code> · <code>ges</code> · <code>lts</code> · <code>les</code> | element-by-element comparison, giving 1 or 0  |
| <code>ifs(control, true.values, false.values)</code>                                                            | picks from one array or the other per element |

| CALL                                                             | RESULT                                           |
|------------------------------------------------------------------|--------------------------------------------------|
| <code>not(0)</code> , <code>not(1)</code> , <code>not('')</code> | <code>1</code> , <code>0</code> , <code>1</code> |
| <code>nots(1\ 0\ '')</code>                                      | <code>0\ 1\ 1</code>                             |
| <code>ands(1\ 0\ 1, 1\ 1\ 0)</code>                              | <code>1\ 0\ 0</code>                             |
| <code>ors(1\ 0\ 0, 0\ 1\ 0)</code>                               | <code>1\ 1\ 0</code>                             |
| <code>eqs(1\ 2\ 3, 1\ 20\ 3)</code>                              | <code>1\ 0\ 1</code>                             |
| <code>ifs(1\ 0, 'yes'\ 'yes', 'no'\ 'no')</code>                 | <code>yes\ no</code>                             |

## Sorting

```
sortinit {order}
sortadd key, data
sortclear
```

`sortinit` starts a sort, `sortadd` feeds it a key and a record, and reading it back drains it. The order codes are the same `a1` / `ar` / `d1` / `dr` as `locate`. For sorting a whole file, `sselect` is faster — see SD Basic - Select Lists.

---

## Mark mapping

```
mark.mapping file.variable, on | off | expression
```

Controls whether SD translates mark characters when reading and writing a **directory** file, whose records are ordinary operating-system files. With mapping on, field marks become newlines on the way out and back again on the way in; with it off, the bytes pass through unchanged.

**It has no effect on a dynamic file**, where records are stored in SD's own format.

---

## What is not here

Nothing in the dynamic-array group has been removed from this port.

`dcount()`, `fieldstore()`, `substrings()`, `splice()`, `substitute()` and the whole `s` family are also described in SD Basic - String Functions, where they are used on text rather than on structure.

---

## See also

SD Basic - String Functions · SD Basic - File Handling · SD Basic - Select Lists · SD Basic - Math Functions.